TIME MANAGEMENT & TASK PRIORITIZATION SYSTEM

Java Swing • OOP • Collections • Threads • JDBC • MySQL

Presentation with illustrative Java code for every topic in the supplied marking rubric.

Note: Code snippets are educational examples; integrate and test them in the complete project.

1. Project Overview & Features

- **Goal:** create, view, prioritize, complete, and grade tasks.
- **GUI:** Java Swing (JFrame, buttons, forms, JTable).
- **Persistence:** MySQL database connected through JDBC.
- **Original menu features retained:** Add Task, View All Tasks, View by Priority, Mark Completed, Grade Task, Exit.
- **Rubric:** OOP (10), Collections & Generics (6), Multithreading & Synchronization (4), database operation classes (7), JDBC connectivity (3), JDBC implementation (3).

```
Task task = new Task("Java assignment", "Medium");
task.markCompleted();
System.out.println(task.getStatus()); // Completed
```

2. OOP: Classes, Encapsulation & Inheritance

Encapsulation keeps fields private; inheritance lets specialized task types reuse the base Task class.

```
class Task {
    private String title;
    private String priority;
    private String status = "Pending";

    Task(String title, String priority) {
        this.title = title;
        this.priority = priority;
    }
    public String getTitle() { return title; }
    public String getPriority() { return priority; }
    public String getStatus() { return status; }
    public void markCompleted() { status = "Completed"; }
}

class StudyTask extends Task {
```

3. OOP: Polymorphism, Interfaces & Exception Handling

```
interface TaskOperations {
    void addTask(Task task);
    java.util.List<Task> getAllTasks();
    void markCompleted(int id) throws java.sql.SQLException;
}

class StudyTask extends Task {
    StudyTask(String title, String priority) { super(title, priority); }
    public String getTaskType() { return "Study"; }
}

try {
    if (title == null || title.trim().isEmpty())
        throw new IllegalArgumentException("Title is required");
    // continue saving the task
} catch (IllegalArgumentException ex) {
```

Polymorphism example: store StudyTask and ProjectTask objects in a List and call overridden methods through the Task reference.

4. Collections & Generics — 6 Marks

```
import java.util.ArrayList;
import java.util.List;

List<Task> tasks = new ArrayList<>();
tasks.add(new Task("Java assignment", "Medium"));
tasks.add(new Task("Revise DBMS", "High"));
```

```
for (Task task : tasks) {
    if (task.getPriority().equalsIgnoreCase("Medium"))
 {
        System.out.println(task.getTitle());
    }
}
```

- **Collection:** ArrayList stores multiple Task objects.
- **Generics:** List<Task> provides type safety.
- **Filtering:** iterate through tasks and compare priority.
- For the final app, database results can be loaded into a List<Task> before refreshing the JTable.

5. Multithreading & Synchronization — 4 Marks

```
private final List<Task> tasks = new ArrayList<>();

public synchronized void addToMemory(Task task) {
    tasks.add(task);
}

// SwingWorker runs work in the background.
new javax.swing.SwingWorker<List<Task>, Void>() {
    @Override
    protected List<Task> doInBackground() throws Exception {
        return taskDAO.getAllTasks(); // database work
    }
    @Override
    protected void done() {
        try {
            List<Task> loaded = get();
            // update JTable on Swing Event Dispatch Thread
            tableModel.setRowCount(0);
```

6. Database Classes & JDBC Connectivity — 10 Marks Combined

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/task_manager";
    private static final String USER = "root";
    // Use a local environment variable for the password.
    public static Connection getConnection() throws SQLExcepti
on {
        String password = System.getenv("DB_PASSWORD");
        return DriverManager.getConnection(URL, USER, password
);
    }
```

- **DBConnection:** opens a JDBC connection.
- **TaskDAO:** owns SQL and CRUD operations.
- Add MySQL Connector/J to the project dependencies.
- Create the database and table before running the app.
- Never upload credentials to GitHub.

7. JDBC Implementation: INSERT & SELECT

```
class TaskDAO {
    public void addTask(String title, String priority)
            throws SQLException {
        String sql = "INSERT INTO tasks(title, priority, status) "
                   + "VALUES (?, ?, ?)";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, title);
            ps.setString(2, priority);
            ps.setString(3, "Pending");
            ps.executeUpdate();
        }
    }

    public List<Task> getAllTasks() throws SQLException {
        List<Task> result = new ArrayList<>();
        String sql = "SELECT title, priority FROM tasks";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
```

8. JDBC Update, SQL Table & Database Operations

```
String sql = "UPDATE tasks SET status = ? WHERE id =
 ?";
try (Connection con = DBConnection.getConnection();
     PreparedStatement ps = con.prepareStatement(sql
)) {
     ps.setString(1, "Completed");
     ps.setInt(2, taskId);
     ps.executeUpdate();
```

```
CREATE DATABASE task_manager;
USE task_manager;
CREATE TABLE tasks (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(120) NOT NULL,
  description TEXT,
  priority VARCHAR(10) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT "P
ending"
  due_date DATE,
  grade VARCHAR(10)
);
```

Use PreparedStatement for SQL parameters and try-with-resources to close connections/results safely. Add updateGrade() and deleteTask() methods if those actions are enabled in the GUI.

9. Java Swing GUI — Main Window Example

```
import javax.swing.*;
import java.awt.*;

public class TaskManagerGUI {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Task Manager");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(700, 450);
            frame.setLayout(new BorderLayout());

            JButton add = new JButton("Add Task");
            JButton view = new JButton("View All Tasks");
            JButton complete = new JButton("Mark Completed");
```

This is a minimal window scaffold. Connect each button to TaskDAO and a table model to complete the working GUI.

10. Workflow, Testing & Exception Cases

- **Add:** validate title and priority → call TaskDAO.addTask() → refresh task table.
- **View:** TaskDAO.getAllTasks() → populate JTable.
- **Filter:** show tasks matching High / Medium / Low.
- **Complete:** update status by task ID and refresh table.
- **Grade:** validate grade input and update the database.
- **Test:** blank title, invalid task ID, database offline, duplicate clicks, and restart persistence.

```
try {
    taskDAO.markCompleted(taskId);
    JOptionPane.showMessageDialog(frame, "Task completed");
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(frame,
        "Database operation failed. Check connection.");
```

11. Marking Rubric Coverage Checklist

OOP: inheritance, polymorphism, exception handling, interfaces	10 marks
Collections & Generics	6 marks
Multithreading & Synchronization	4 marks
Database operation classes	7 marks
Database Connectivity (JDBC)	3 marks
JDBC implementation	3 marks

Total listed rubric: 33 marks. This checklist describes what the source code must actually implement; slides alone do not earn implementation marks.

12. Setup & GitHub Submission

- Install a compatible JDK and configure VS Code Java support.
- Install MySQL and create the task_manager database/table using the SQL slide.
- Add the MySQL Connector/J dependency and set DB_PASSWORD locally.
- Run and test each class; resolve package names and imports consistently.
- Upload source files, SQL setup script, and README to GitHub.
- Do not upload secrets. Add screenshots only after the GUI actually runs.
- Submit the PDF presentation and the correct repository URL through the required form.

THANK YOU

Time Management & Task Prioritization System

The presentation now includes Java code examples for the rubric topics: OOP, collections, concurrency, database classes, JDBC, and Swing GUI.

Important: these are code examples and scaffolding, not proof that the complete integrated application has already been implemented. Test and integrate the snippets before final submission.